

Week 7 — Lab Handout (Student)

Binding Forms to the Backend

Time: ~90 min in-class **Work as:** Your team (5 members) **AI use this week:** ✓ **ON — required.** Scaffold, review, and **log your prompts.** **What you leave with:** Working Create and Update forms that send data to your controllers and handle loading/success/error.

Why this lab matters

Today your app becomes real. The views from Week 6 start sending data to the controllers from Phase 2 — click "Save" in the UI and a record actually persists. You'll also handle what happens while you wait (loading) and when things go wrong (errors), making the empty/loading/error states you wireframed finally do their job.

Task 1 — Wire the Create form (25 min)

Bind your "Create" form to the `POST` endpoint from Phase 2.

```
on submit:
  e.preventDefault()                # stop the full-page reload
  data = collect form fields
  button.disable(); showSpinner()    # loading state
  response = await send(POST, "/orders", data)
  if response.status == 201: addToList(response.data); resetForm()
  else if response.status == 422: showFieldErrors(response.error)
  button.enable(); hideSpinner()
```

- ☐ Submitting the form sends data to your `POST` controller (no full-page reload).
- ☐ A new record actually persists and appears in the UI.
- ☐ The submit button is **disabled while pending** (prevents double-submits).

Task 2 — Wire the Update form (20 min)

Bind your "Edit" form to the `PUT` endpoint.

- ☐ The edit form is **pre-filled** with the existing record's values.
- ☐ Submitting sends changes to your `PUT` controller.
- ☐ The updated values are reflected in the UI after success.

Task 3 — Handle the async lifecycle (20 min)

Make the loading/success/error states real, using the standardized responses your controllers return.

- ☐ **Loading** — spinner/disabled button while the request is pending.
- ☐ **Success** — UI updates; form clears or closes; optional confirmation.
- ☐ **Error (422)** — show the field-level error messages from the response.
- ☐ **Error (500/network)** — show a general failure message, don't crash the UI.

If a failed submit looks like nothing happened, your error handling is missing. Every failure must be visible.

Task 4 — Test end to end (15 min)

- ☐ Create a record through the UI → it persists and shows up.
- ☐ Edit that record through the UI → the change persists.
- ☐ Submit invalid data → the 422 field errors display.
- ☐ Record the test results in `/docs/binding-tests.md`.

Task 5 — Use AI well, and log it (10 min)

- ☐ Use AI to scaffold fetch/async logic, then review and fix it.

- ☐ Record key prompts in `/docs/ai-notes/week-07.md`, marking AI-generated vs. hand-written.
 - ☐ Each member owns at least one binding on the board; merges via reviewed PR.
-

End-of-Lab Checklist

- ☐ Create form bound to `POST`; records persist; button disabled while pending
 - ☐ Update form pre-filled and bound to `PUT`
 - ☐ Loading, success, and error (422 + general) states all handled and visible
 - ☐ `/docs/binding-tests.md` — end-to-end test results
 - ☐ `/docs/ai-notes/week-07.md` — prompt log with labels
 - ☐ Work owned on the board; all merges via reviewed PR
-

Looking ahead to Week 8

You'll polish **error handling & user feedback** across the whole app — graceful failures, helpful messages, loading indicators everywhere — and close **Deliverable 3 (Interface & View Binding, 30%)**, a fully interactive local application where CRUD works seamlessly.